

COMPASS_Synop- tic_LE_Tree_Flux_Calcs_2023_CO2

Roberta Peixoto

2026-04-23

Table of contents

1	Add Required Packages	2
2	Pull in Raw Datafiles	2
3	Read in metadata file and match to raw data	4
4	Check start times overlaps or missing values	5
5	Match metadata to raw data	6
6	Temperature from weather station	8
7	fixing some Deadbands	8
8	Calculate fluxes	10
9	check the fluxes for which I changed the start time due to lags or high values (recorded in notes of metadata)	17
10	Spot check a few fluxes with $R^2 > 0.9$	18
11	Take a look at the fluxes < 0.9 and a flux estimate below 1	23
12	Take a look at the bad fluxes < 0.9 and have a high flux estimate	26
13	See if we want to remove these fluxes or if we want to try and recalculate these ones?	26

14 Look at the really high or really low ones to see if there is a timing issue or something	30
15 Check all fluxes	32
16 Remove any fluxes that do not fit criteria	35
17 Do some visualization of QAQC plots to assess fluxes	36
18 Overall checks for r2	38
19 Read out final dataframe and export for use elsewhere	39

1 Add Required Packages

2 Pull in Raw Datafiles

```
#defining where the raw data is

DATA_DIR_ROOT <- here::here(
  "Tree_Fluxes_Synoptic_LE",
  "2023",
  "Raw Data"
)

#function to read data
now <- function() format(Sys.time(), "%Y-%m-%d %H:%M:%S")
message(now(), " | Script started")
#finding the zip files
message(now(), " | Looking for ZIP files in: ", DATA_DIR_ROOT)

zip_files <- list.files(
  DATA_DIR_ROOT,
  pattern = "\\\\.zip$",
  full.names = TRUE)

message(now(), " | ZIP files found: ", length(zip_files))
print(zip_files)
if (length(zip_files) == 0) {
```

```

    stop("No ZIP files found. Check DATA_DIR_ROOT.")}

cache <- list()

# Reader function
read_li7810_zip <- function(zip_file, tz = "US/Eastern") {
  message(now(), " | Processing ZIP: ", basename(zip_file))
  if (zip_file %in% names(cache)) {
    message("  -> Using cached version")
    return(cache[[zip_file]]) }

  tmp_dir <- tempfile("li7810_")
  dir.create(tmp_dir)
  on.exit(unlink(tmp_dir, recursive = TRUE), add = TRUE)
  unzip(zip_file, exdir = tmp_dir)
  data_files <- list.files(
    tmp_dir,
    pattern = "\\\\.data$",
    full.names = TRUE,
    recursive = TRUE )

  message("  -> .data files found: ", length(data_files))
  if (length(data_files) == 0) {
    stop("No .data files inside ZIP: ", zip_file) }

  dat_list <- lapply(data_files, function(f) {
    message("    Reading: ", basename(f))
    ffi_read_LI7810(f)})

  dat <- data.table::rbindlist(dat_list, fill = TRUE) %>%
    mutate(TIMESTAMP = with_tz(TIMESTAMP, tz))
  cache[[zip_file]] <- dat
  message("  -> ZIP finished: ", basename(zip_file))
  return(dat)}

# ACTUAL EXECUTION
message(now(), " | Starting data read")
alldat_list <- lapply(zip_files, read_li7810_zip)
alldat <- data.table::rbindlist(alldat_list, fill = TRUE)
message(now(), " | DONE")
message("Final number of rows: ", nrow(alldat))

```

```
# Show result
print(head(alldat))
```

3 Read in metadata file and match to raw data

```
md <- "Raw Data/COMPASS_SynopticLE_TreeFlux_Metadata_2023.csv"
dat <- read.csv(md)

metadat <- dat %>%
  # Remove rows without date
  filter(!is.na(Date)) %>%
  # Keep only desired instruments
  filter(SN %in% c("TG10-01448", "TG10-01426", "TG10-01028")) %>%
  # Normalize Plot labels
  mutate(
    Plot_raw = toupper(Plot),
    # Extract plot components
    #Site = str_extract(Plot_raw, "[A-Z]{3}"),
    #Zone = str_extract(Plot_raw, "(TR|UP)"), # somehow all of the PTR measurements turned i
    Site = toupper(Site),
    Zone = toupper(Zone),
    Replicate = str_extract(Plot_raw, "(?<=TR|UP)[0-9]+"),
    # Letter ONLY if it comes AFTER a number
    Letter = str_extract(Plot_raw, "(?<=[0-9])[A-Z]$"),
    # Time normalization
    Time_start = str_replace_all(as.character(Time_start), "\\.", ":") |> str_trim(),
    Stop.time = str_replace_all(as.character(Stop.time), "\\.", ":") |> str_trim(),
    # Safe time parsing (NO WARNINGS)
    t_start = suppressWarnings(parse_date_time(Time_start, orders = c("H:M:S", "H:M"))),
    t_stop = suppressWarnings(parse_date_time(Stop.time, orders = c("H:M:S", "H:M"))),
    # Observation length
    Obs_length = as.numeric(t_stop - t_start, units = "secs"),
    # Timestamp
    TIMESTAMP = as.POSIXct(
      paste(Date, Time_start),
      format = "%m/%d/%Y %H:%M:%S",
      tz = "EST"
    ),
    # Sequential ID
```

```

Seq = row_number(),
# INAL standardized Plot label
Plot = if_else(
  is.na(Letter),
  paste(Date, Site, Zone, Replicate, Seq, sep = "_"),
  paste(Date, Site, Zone, Replicate, Letter, Seq, sep = "_")) )

# Quick check
head(metadata$Plot)

```

4 Check start times overlaps or missing values

```

#checking if there is any start and stop time overlap from the field notes
overlaps_detected <- metadata %>%
  mutate(
    start_ts = as.POSIXct(
      paste(Date, Time_start),
      format = "%m/%d/%Y %H:%M:%S",
      tz = "EST"
    ),
    end_ts = start_ts + Obs_length
  ) %>%
  arrange(SN, start_ts) %>%
  group_by(SN) %>%
  mutate(
    next_plot      = lead(Plot),
    next_start_ts  = lead(start_ts),
    overlap_fffi   = !is.na(next_start_ts) & next_start_ts <= end_ts
  ) %>%
  filter(overlap_fffi) %>%
  select(
    SN,
    Plot,
    start_ts,
    end_ts,
    next_plot,
    next_start_ts,
    Obs_length
  )

```

```

print(overlaps_detected)

#checking problems with obs_length - this is a frequent bug for manual field notes
field_notes_overlap<-metadat %>%
  mutate(
    obs_problem =
      is.na(Obs_length) | Obs_length <= 0
  ) %>%
  filter(obs_problem) %>%
  select(
    SN,
    Date,
    Plot,
    Time_start,
    Stop.time,
    Obs_length
  )

print(field_notes_overlap)

# checking unique measurment start time and stop time
# alldat %>%
#   filter(
#     SN == "TG10-01448",
#     TIMESTAMP > as.POSIXct("2023-01-05 13:28:00"),
#     TIMESTAMP < as.POSIXct("2023-01-05 13:33:00")
#   ) %>%
#   ggplot(aes(x = TIMESTAMP, y = CO2)) +
#   geom_point()#ylim(300,500)

```

5 Match metadata to raw data

```

#function to match data per equipment (considering different serial numbers) uhul!
match_flux_metadata <- function(alldat, metadat) {
  message("Starting metadata Licor matching")
  #Sanity checks (fail early)
  stopifnot(
    "TIMESTAMP" %in% names(alldat),
    "SN" %in% names(alldat),

```

```

    "Date"          %in% names(metadat),
    "Time_start"    %in% names(metadat),
    "Obs_length"    %in% names(metadat),
    "Plot"          %in% names(metadat)
  )

  stopifnot(all(metadat$Obs_length > 0, na.rm = TRUE))
  stopifnot(anyDuplicated(metadat$Plot) == 0)
# Instruments present in BOTH datasets
instruments <- intersect(
  unique(alldat$SN),
  unique(metadat$SN)
)
message("Instruments to process: ", paste(instruments, collapse = ", "))
results <- lapply(instruments, function(sn) {
  message("Processing SN: ", sn)
  dat_i <- alldat %>%
    filter(SN == sn)
  meta_i <- metadat %>%
    filter(SN == sn) %>%
    arrange(Date, Time_start) %>%
    mutate(metadat_row = row_number())
  stopifnot(nrow(meta_i) > 0)
# Actual matching
dat_i$metadat_row <- ffi_metadata_match(
  data_timestamps = dat_i$TIMESTAMP,
  start_dates     = meta_i$Date,
  start_times     = meta_i$Time_start,
  obs_lengths     = meta_i$Obs_length )
# Attach metadata (vectorized, fast)
dat_i <- dat_i %>%
  mutate(
    Plot      = meta_i$Plot[metadat_row],
    Volume    = meta_i$Volume[metadat_row],
    Temp      = meta_i$Temp[metadat_row],
    Area      = meta_i$Area[metadat_row],
    Dead_band = meta_i$Dead_band[metadat_row]
  )
# Drop unmatched rows
n_before <- nrow(dat_i)
dat_i <- dat_i %>% filter(!is.na(metadat_row))
n_after <- nrow(dat_i)
message("  Matched rows: ", n_after, " / ", n_before)

```

```

    dat_i })
  alldat_matched <- rbindlist(results, fill = TRUE)
  #Final sanity checks
  stopifnot(
    !anyNA(alldat_matched$Plot),
    nrow(alldat_matched) > 0
  )
  message("Matching finished successfully")
  return(alldat_matched)}

#running the function
message("Running final metadata matching...")
alldat_matched <- match_flux_metadata(alldat, metadat)
message("Matched dataset rows: ", nrow(alldat_matched))
head(alldat_matched)

#saving matched data and metadata as raw data is too big (today, Licor files leave in the CO2 column)
saveRDS(alldat_matched, file = "alldat_matched_2023.rds")

#can remove any row that have NA's in the CO2 column
#alldat <- alldat %>%
# filter(!is.na(CO2))

##Reading matched data and metadata

# after you match the data for the first time - we can just read the .rds file to save time
alldat_matched <- readRDS("alldat_matched_2023.rds")

```

6 Temperature from weather station

```

# I need to go back here to get temp from the weather station

```

7 fixing some Deadbands


```

# Table of plots with custom deadband adjustments
deadband_overrides <- tibble::tibble(
  Plot = c(
    "7/13/2023CRC_TR315-115",
    "7/27/2023PTR_UP8-125",
    "7/13/2023CRC_TR315-115",
    "3/28/2023OWC_TR127-170",
    "5/10/2023OWC_UP103-73",
    "7/13/2023CRC_TR314-114",
    "8/3/2023CRC_TR314-133",
    "7/13/2023CRC_TR316-116",
    "6/29/2023CRC_TR311-104"

  ),
  DeadBand_override = c(
    8,
    9,
    22,
    22,
    16,
    8,
    16,
    3,
    34

  )
)

# Default deadband
alldat_matched <- alldat_matched %>%
  mutate(
    DeadBand_used = as.numeric(Dead_band),
    DeadBand_used = if_else(
      is.na(DeadBand_used),
      0,
      DeadBand_used
    )
  )

# Apply per-plot overrides
alldat_matched <- alldat_matched %>%
  left_join(deadband_overrides, by = "Plot") %>%
  mutate(

```

```

DeadBand_used = if_else(
  !is.na(DeadBand_override),
  DeadBand_override,
  DeadBand_used
)
) %>%
select(-DeadBand_override)

```

8 Calculate fluxes

```

# TO Do - NEED TO PULL TEMP from the weather station
#change the units
alldat_matched$CO2_umol <- ffi_ppb_to_nmol(alldat_matched$CO2,
  volume = alldat_matched$Volume, # m3
  temp = alldat_matched$Temp)    # degrees C

```

Assuming atm = 101325 Pa

Using R = 8.31446261815324 m3 Pa K-1 mol-1

```

#> Assuming atm = 101325 Pa
#> Using R = 8.31446261815324 m3 Pa K-1 mol-1

alldat_matched$CO2_umol_m2 <- alldat_matched$CO2_umol / 0.16

fluxes <- ffi_compute_fluxes(alldat_matched ,
  group_column = "Plot",
  time_column = "TIMESTAMP",
  gas_column = "CO2_umol_m2",
  dead_band = "DeadBand_used")

```

NOTE: HM81_flux.estimate is not NA, implying nonlinear data

NOTE: HM81_flux.estimate is not NA, implying nonlinear data

NOTE: HM81_flux.estimate is not NA, implying nonlinear data

NOTE: HM81_flux.estimate is not NA, implying nonlinear data

NOTE: HM81_flux.estimate is not NA, implying nonlinear data

NOTE: HM81_flux.estimate is not NA, implying nonlinear data

[illegible]

[illegible]

[illegible]

[illegible]

NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data

```
head(fluxes)
```

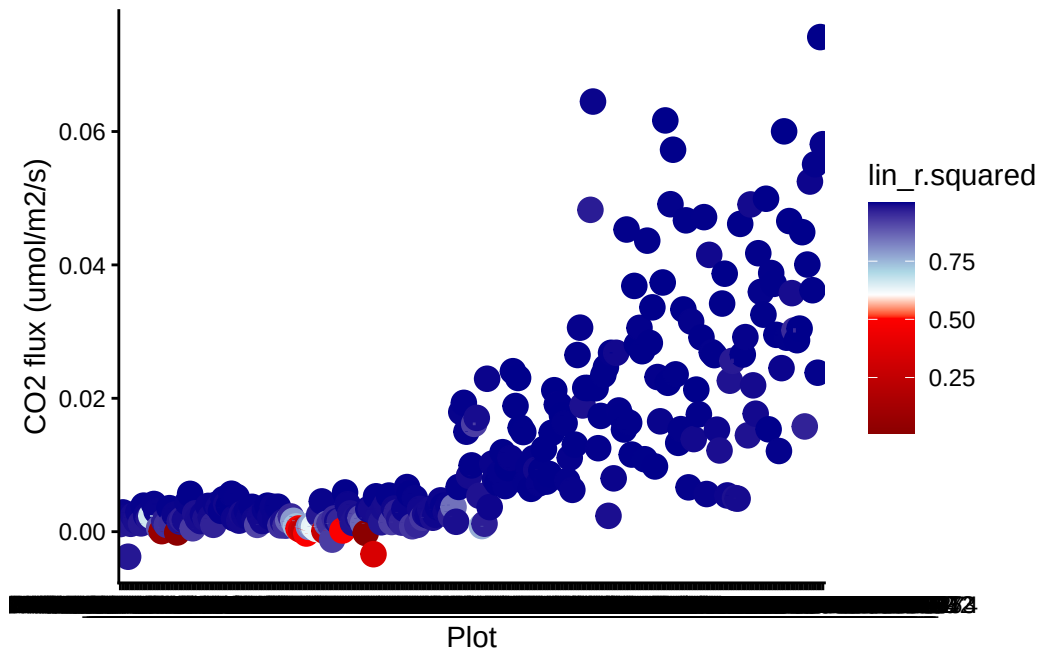
	Plot	TIMESTAMP	HM81_AIC	HM81_flux.estimate
1	1/12/2023_PTR_TR_1_26	2023-01-12 11:25:29	-1756.7676	0.001388476
2	1/12/2023_PTR_TR_2_27	2023-01-12 11:41:49	-483.3205	0.003585076
3	1/12/2023_PTR_TR_3_28	2023-01-12 11:55:29	-961.6335	0.002174485
4	1/12/2023_PTR_TR_4_29	2023-01-12 12:07:29	-390.4527	-0.006328424
5	1/12/2023_PTR_TR_4_B_30	2023-01-12 12:18:29	-1665.1006	0.001398955
6	1/12/2023_PTR_TR_5_31	2023-01-12 12:26:29	-894.6177	0.003175208

	HM81_p.value	HM81_r.squared	HM81_RMSE	lin_AIC	lin_flux.estimate
1	7.666169e-156	0.6938552	0.05578546	-3130.103	0.001144084
2	4.580077e-198	0.6957943	0.17560074	-3687.642	0.002892026
3	1.274356e-142	0.6610713	0.10821554	-3393.295	0.002136485
4	7.782742e-168	0.7208468	0.17418343	-1860.846	-0.003754403
5	1.376542e-143	0.6635825	0.06021385	-3234.801	0.001181948
6	4.004211e-161	0.7060499	0.11443094	-2836.655	0.002418971

	lin_flux.std.error	lin_int.estimate	lin_int.std.error	lin_p.value
1	8.373178e-06	14.58246	0.001446651	0
2	7.053247e-06	14.23356	0.001544379	0
3	6.724163e-06	14.42667	0.001161747	0
4	2.411292e-05	17.91397	0.004166038	0

5	7.673598e-06	14.46237	0.001325783	0		
6	1.069283e-05	14.36606	0.001847422	0		
	lin_r.squared	lin_RMSE	poly_AIC	poly_r.squared	poly_RMSE	rob_AIC
1	0.9689635	0.01776209	-3284.189	0.9761523	0.01559583	-3129.778
2	0.9955116	0.02132983	-4242.333	0.9978481	0.01478876	-3685.296
3	0.9941114	0.01426402	-3404.486	0.9942586	0.01410820	-3393.067
4	0.9759267	0.05115095	-2651.145	0.9935938	0.02643107	-1860.834
5	0.9754138	0.01627807	-3303.361	0.9782143	0.01534865	-3234.636
6	0.9884500	0.02268280	-3309.414	0.9947822	0.01527141	-2836.163
	rob_converged	rob_flux.estimate	rob_flux.std.error	rob_RMSE		
1	TRUE	0.001139642	9.114405e-06	0.01913327		
2	TRUE	0.002902213	7.307146e-06	0.01998759		
3	TRUE	0.002139352	7.130646e-06	0.01438402		
4	TRUE	-0.003753151	2.486670e-05	0.06464943		
5	TRUE	0.001181244	7.609461e-06	0.01491134		
6	TRUE	0.002413608	1.098255e-05	0.02395434		
	TIMESTAMP_min		TIMESTAMP_max			
1	2023-01-12 11:23:00	2023-01-12 11:27:59				
2	2023-01-12 11:38:40	2023-01-12 11:44:59				
3	2023-01-12 11:53:00	2023-01-12 11:57:59				
4	2023-01-12 12:05:00	2023-01-12 12:09:59				
5	2023-01-12 12:16:00	2023-01-12 12:20:59				
6	2023-01-12 12:24:00	2023-01-12 12:28:59				

```
#take a look at the fluxes we calculated with R2
ggplot(fluxes, aes(Plot, lin_flux.estimate, color = lin_r.squared)) +
  geom_point(size=4) + theme_classic() +
  geom_linerange(aes(ymin = lin_flux.estimate- lin_flux.std.error,
                    ymax = lin_flux.estimate+ lin_flux.std.error)) +
  scale_color_gradientn(colours = c("darkred", "red", "white", "lightblue", "darkblue"),
                        values = c(0, 0.5, 0.6, 0.7, 1)) +
  ylab("CO2 flux (umol/m2/s)")
```

```
#sort fluxes by R2
fluxes_good <- as.data.frame(subset(fluxes, lin_r.squared >= .90, select = Plot:TIMESTAMP_max)
fluxes_lowr2 <- as.data.frame(subset(fluxes, lin_r.squared < .90 & lin_flux.estimate < 1, select = Plot:TIMESTAMP_max)
fluxes_weird <- as.data.frame(subset(fluxes, lin_r.squared < .90, select = Plot:TIMESTAMP_max)

#find fluxes that represent the first and fourth quartile of the data
low <- quantile(fluxes$lin_flux.estimate, 0.05)
high <- quantile(fluxes$lin_flux.estimate, 0.95)

fluxes_low <- as.data.frame(subset(fluxes, lin_flux.estimate <= low, select = Plot:TIMESTAMP_max)
fluxes_high <- as.data.frame(subset(fluxes, lin_flux.estimate >= high, select = Plot:TIMESTAMP_max)

tails <- rbind(fluxes_low, fluxes_high)
```

9 check the fluxes for which I changed the start time due to lags or high values (recorded in notes of metadata)

```
#### If something is in a V shape should we take the second part to make a positive linear c
# Subset data and create relative time (seconds since flux start)
```

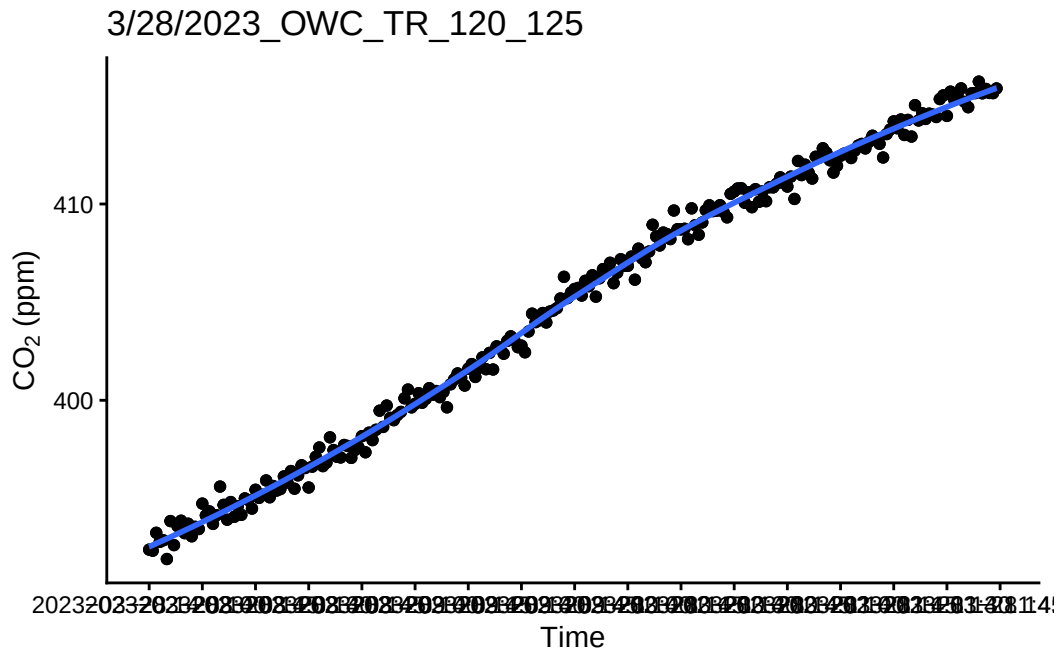
```

# r_all <- alldat %>%
#   filter(Plot %in% deadband_overrides$Plot) %>%
#   group_by(Plot) %>%
#   mutate(
#     time_sec = as.numeric(
#       difftime(TIMESTAMP, min(TIMESTAMP), units = "secs")
#     )
#   ) %>%
#   ungroup()
#
#
# # Random sample of plots for visualization
# plots_sample <- sample(unique(r_all$Plot), 6, replace = FALSE)
#
# r_plot <- r_all %>%
#   filter(Plot %in% plots_sample)
#
# # Plot
# ggplot(
#   r_plot,
#   aes(
#     x = TIMESTAMP,
#     y = co2,
#     color = time_sec >= DeadBand_used
#   )
# ) +
#   geom_point(size = 1.8) +
#   facet_wrap(~ Plot_ID, scales = "free", ncol = 2) +
#   scale_color_manual(
#     values = c("grey60", "red"),
#     labels = c("Discarded (deadband)", "Used after deadband")
#   ) +
#   theme_classic() +
#   ylab(expression(paste(CO[2], " (ppm)"))) +
#   xlab("Time")

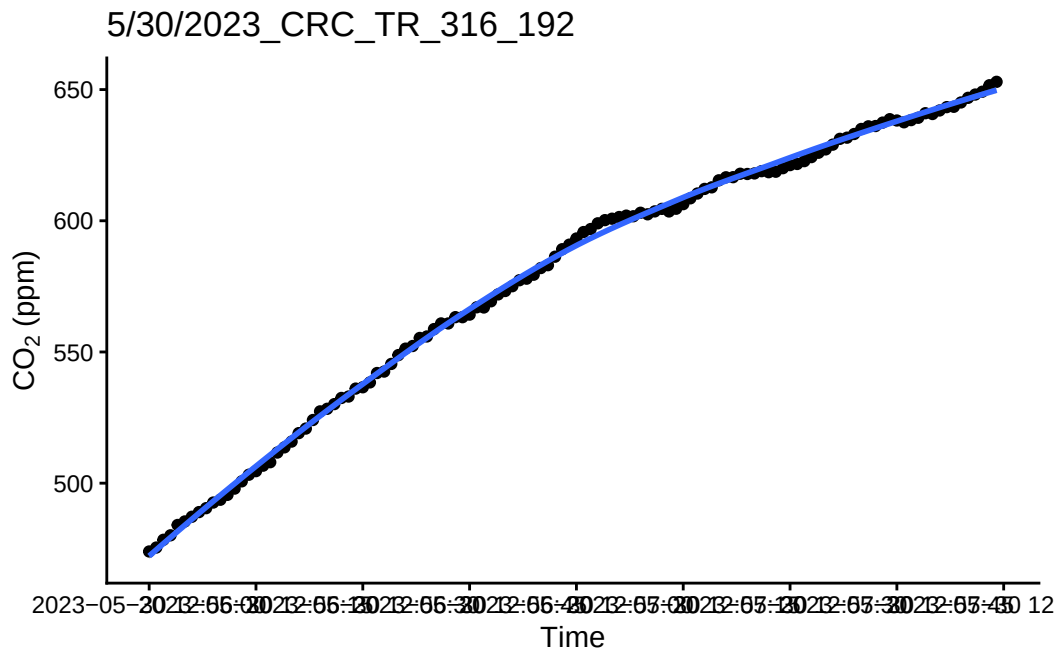
```

10 Spot check a few fluxes with $R2 > 0.9$

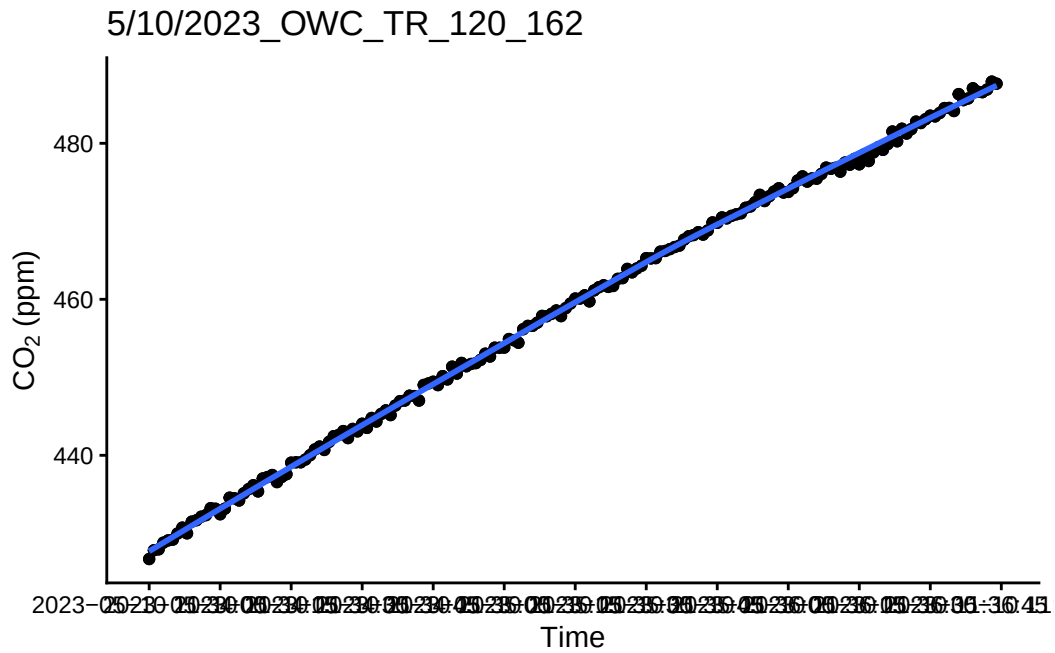
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



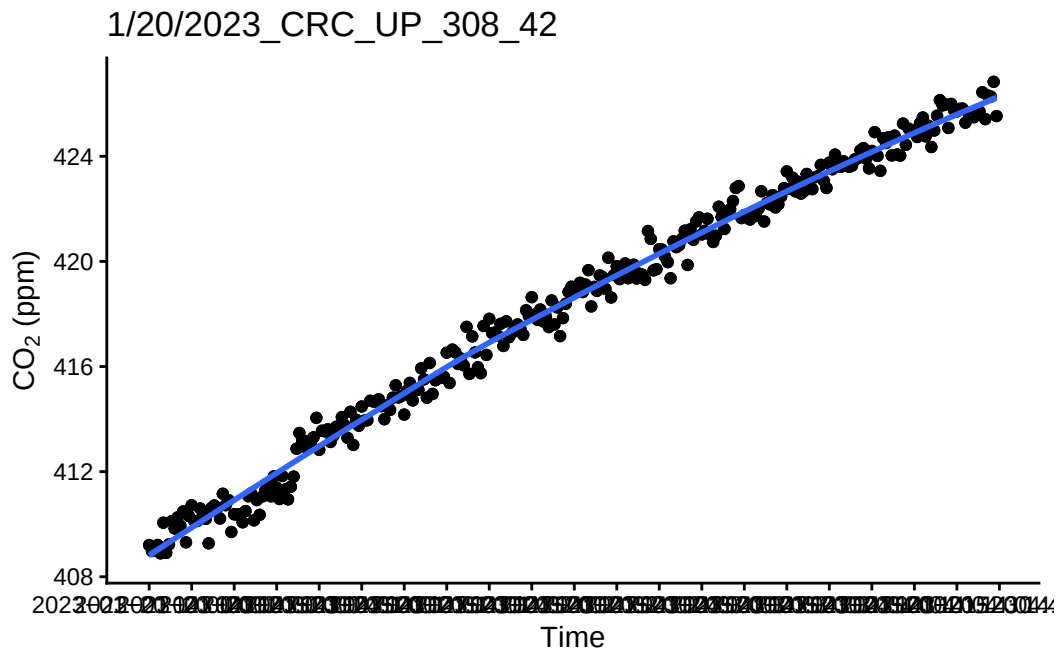
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



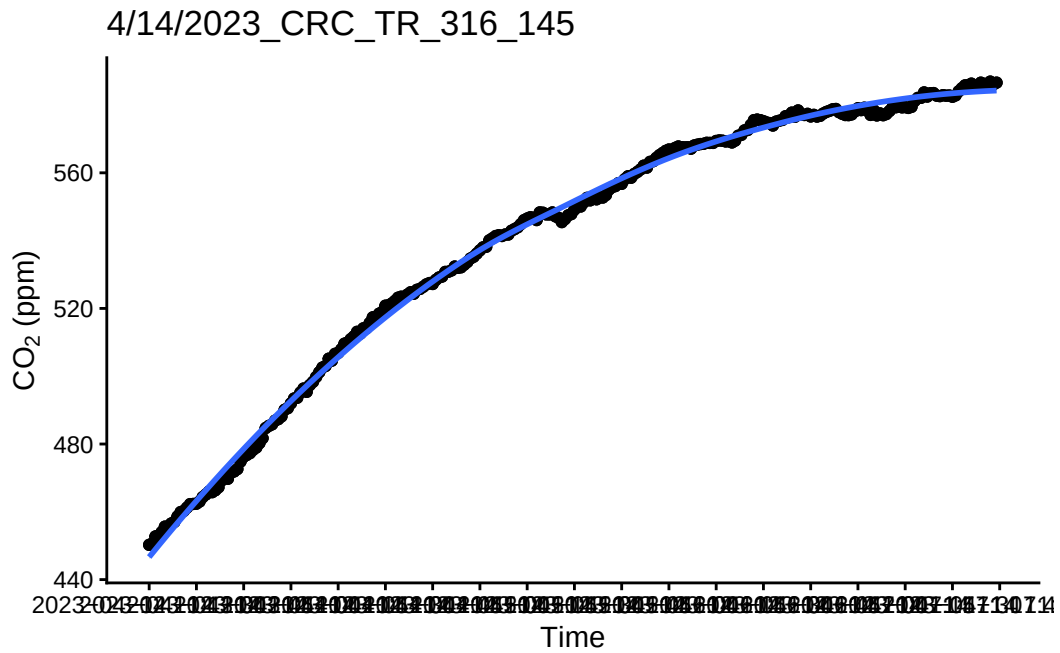
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



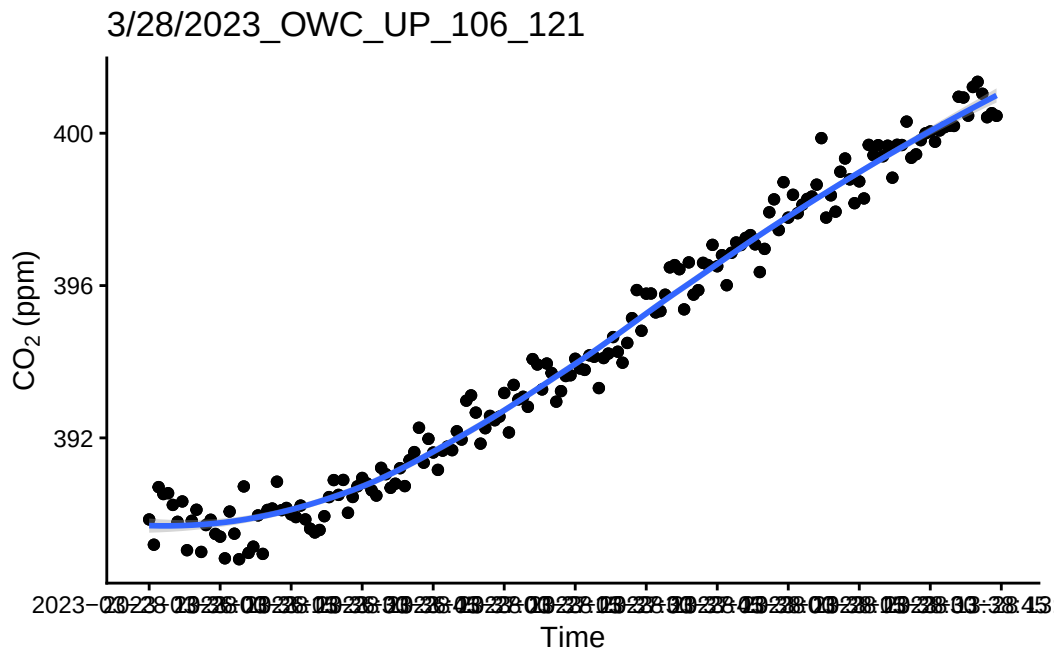
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



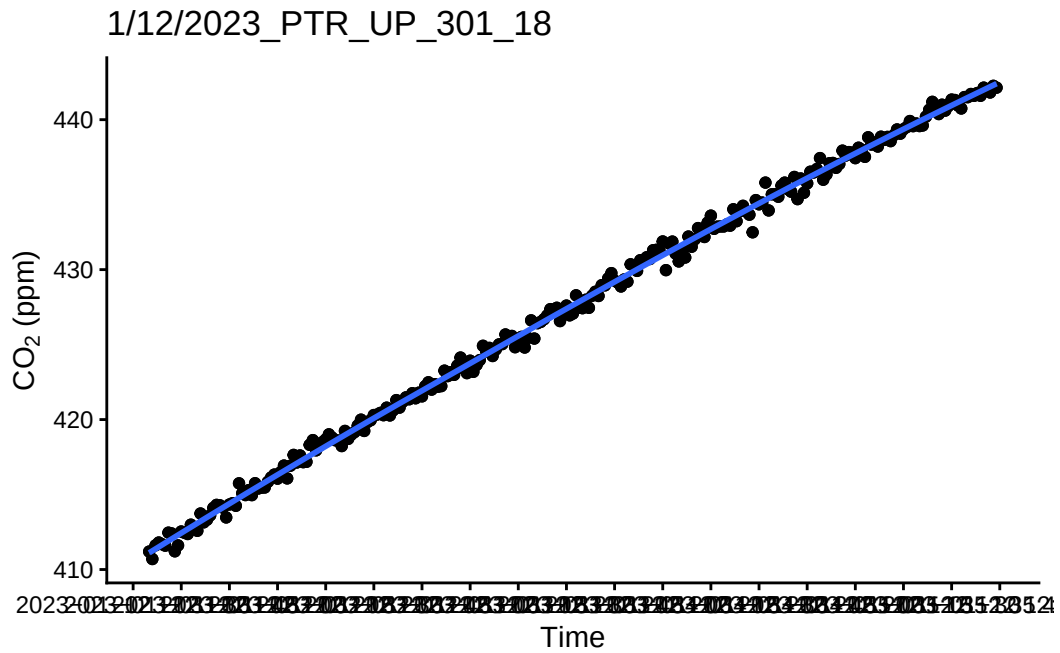
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



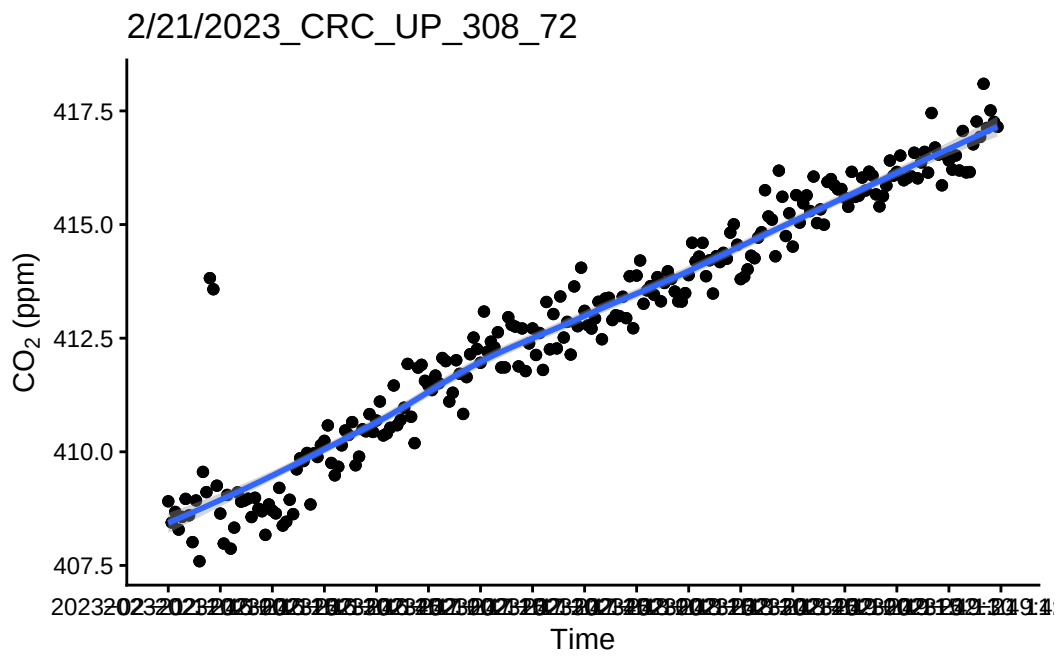
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



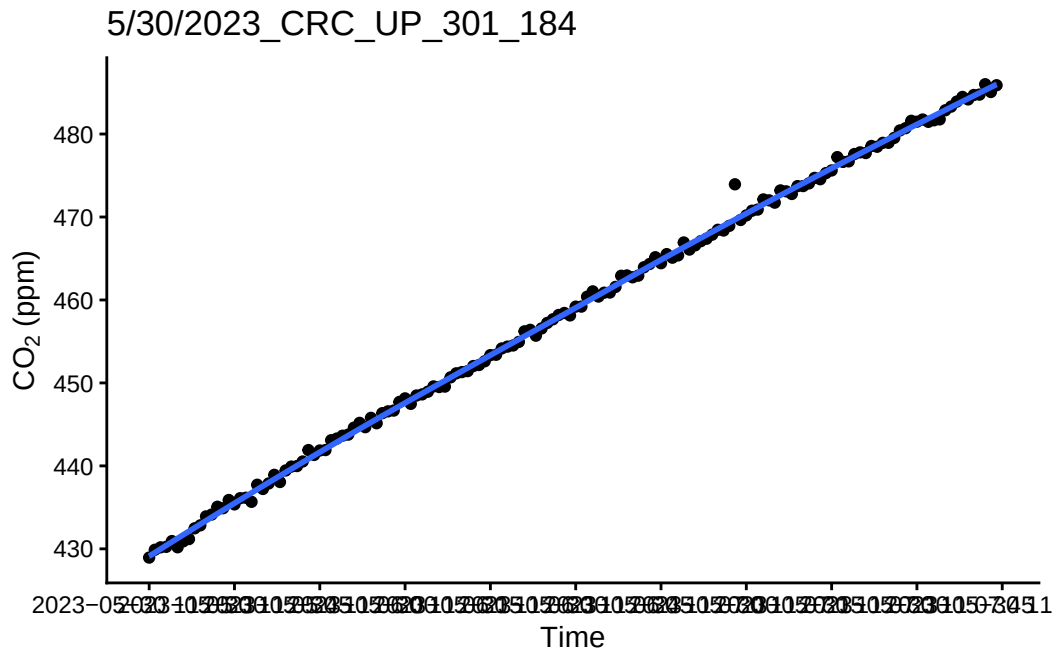
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

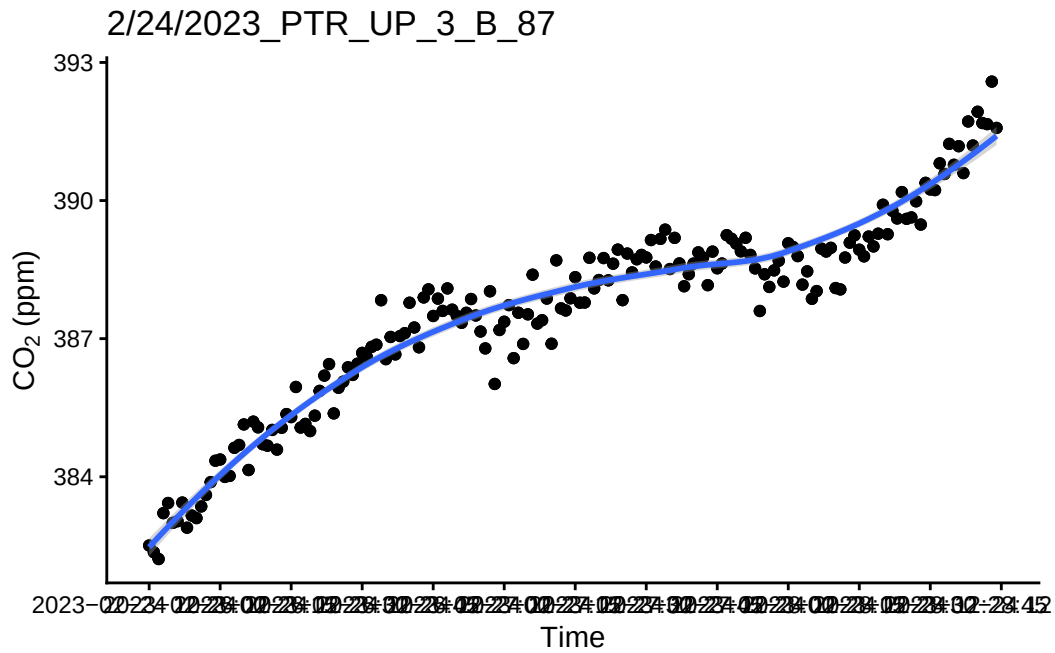


``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

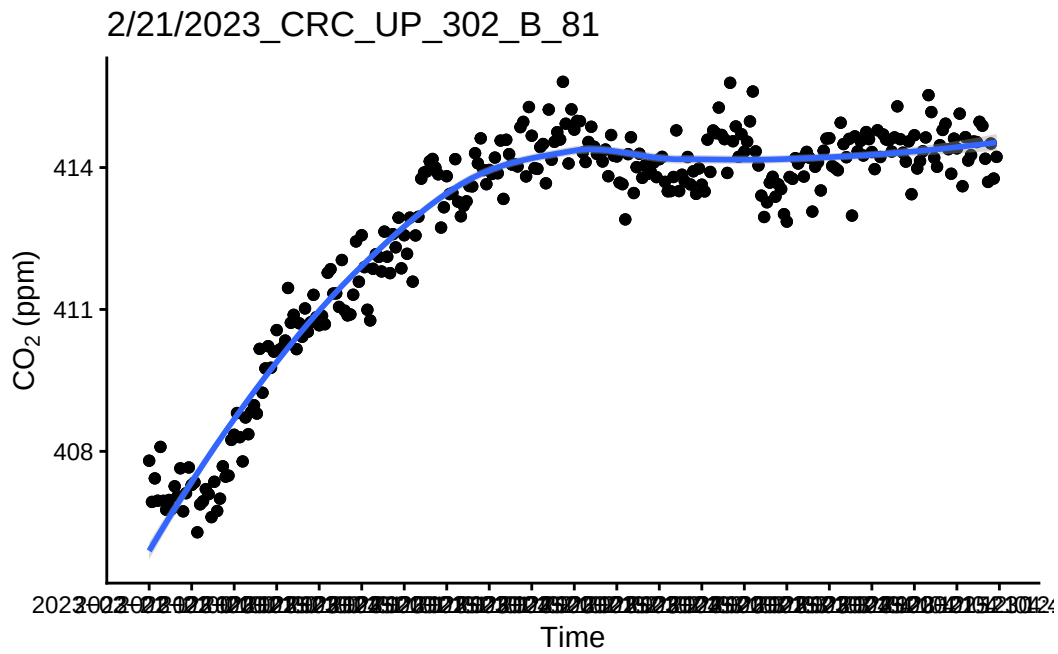


11 Take a look at the fluxes <0.9 and a flux estimate below 1

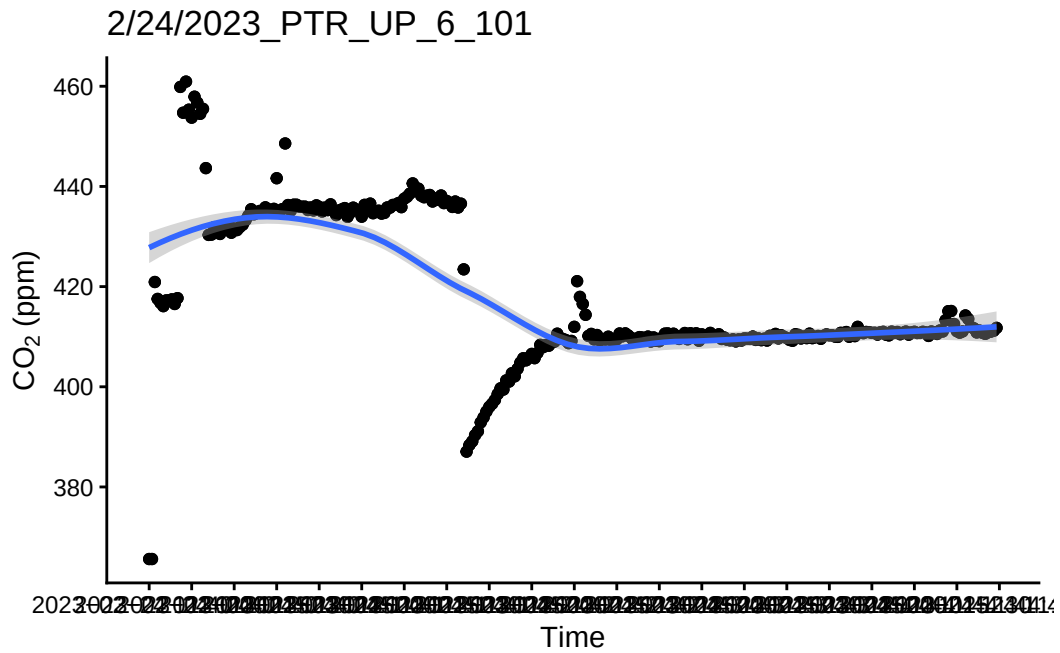
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



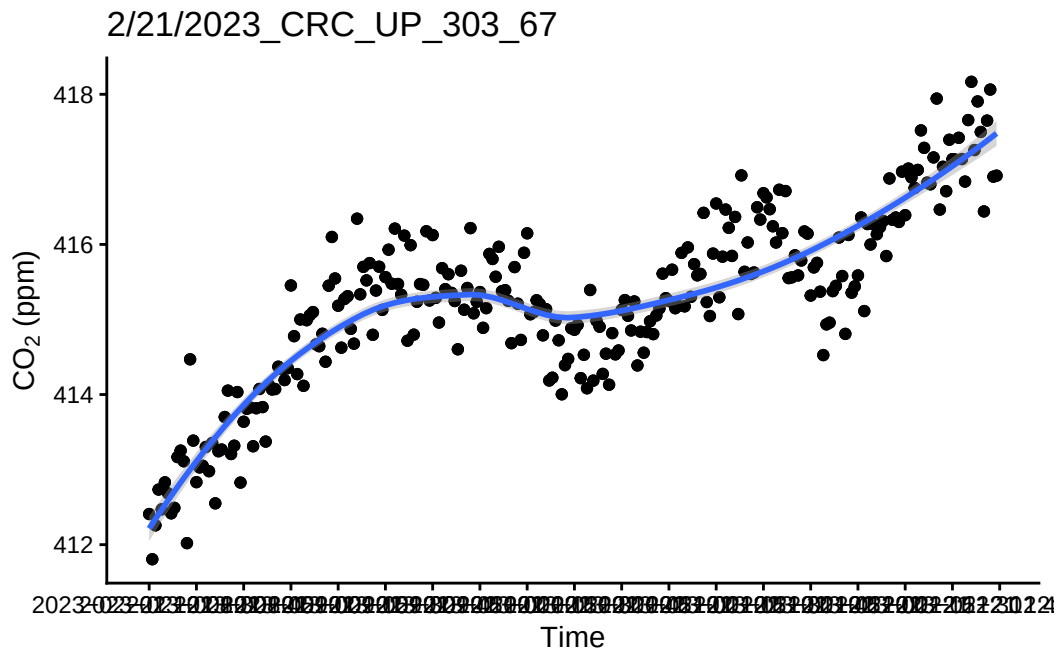
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



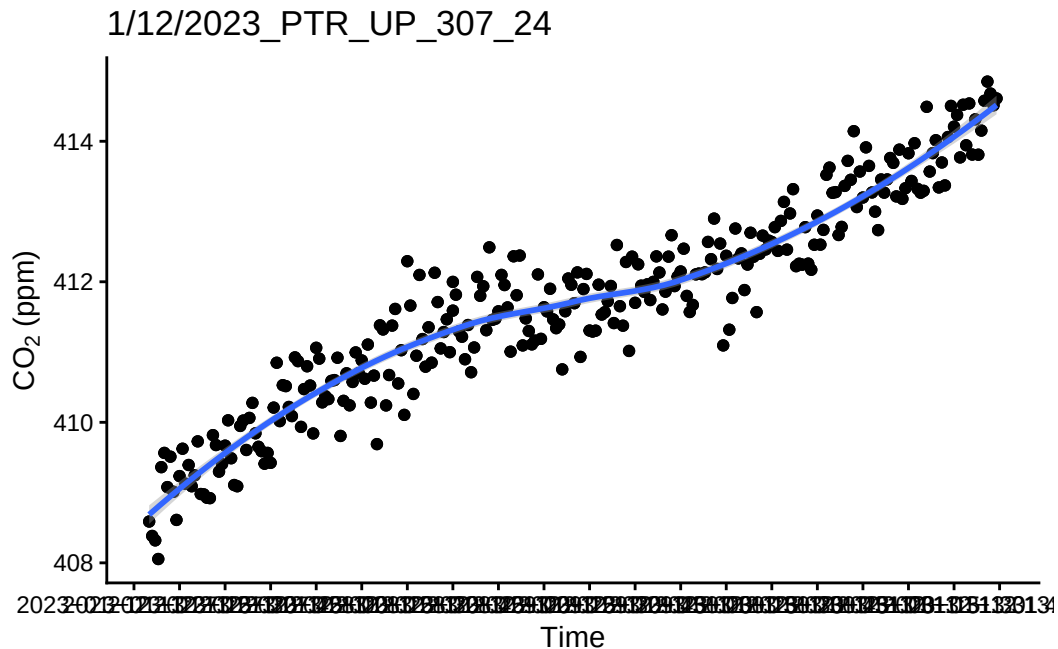
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



12 Take a look at the bad fluxes <0.9 and have a high flux estimate

13 See if we want to remove these fluxes or if we want to try and recalculate these ones?

```
### pull out fluxes with R2 < 0.9 and put them in a list
bad_plots <- unique(fluxes_weird$Plot)

# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
bad_sample <- sample(bad_plots, 5, replace=FALSE)

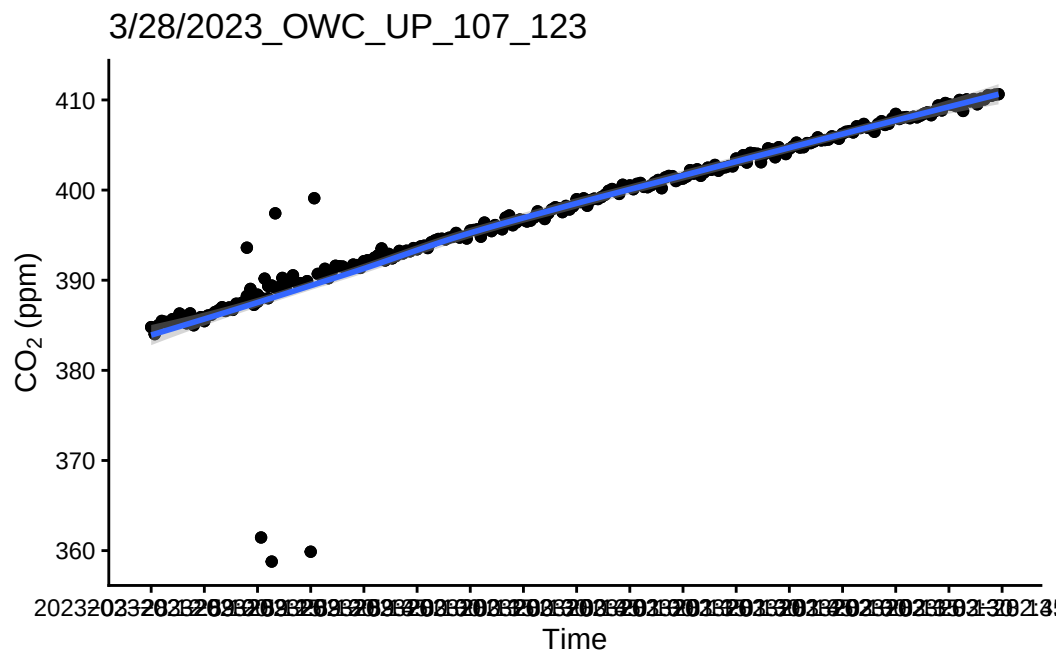
#Loop to run five random plots subset above
for(P in bad_sample){
  r1 <- subset(alldat_matched , Plot == P & !is.na(metadat_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CO2)) + geom_point() + theme_classic() +
    scale_x_datetime(breaks = "15 sec") +
    ylab(expression(paste( CO [2], " (ppm)")) +
```

```

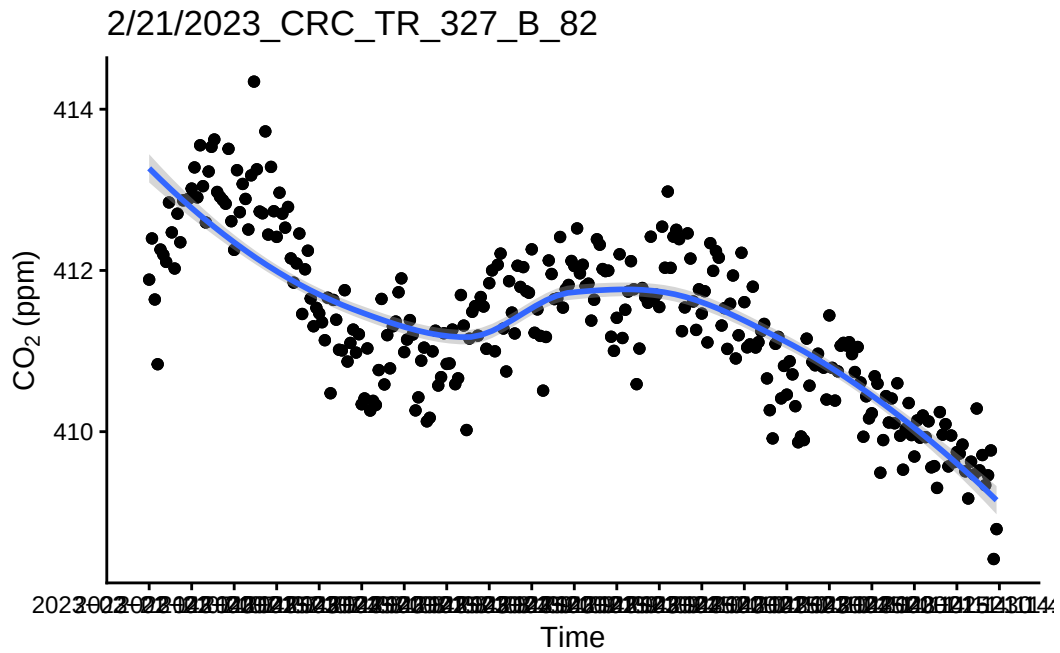
xlab("Time") + labs(title = P) +
geom_smooth()
print(p1)
}

```

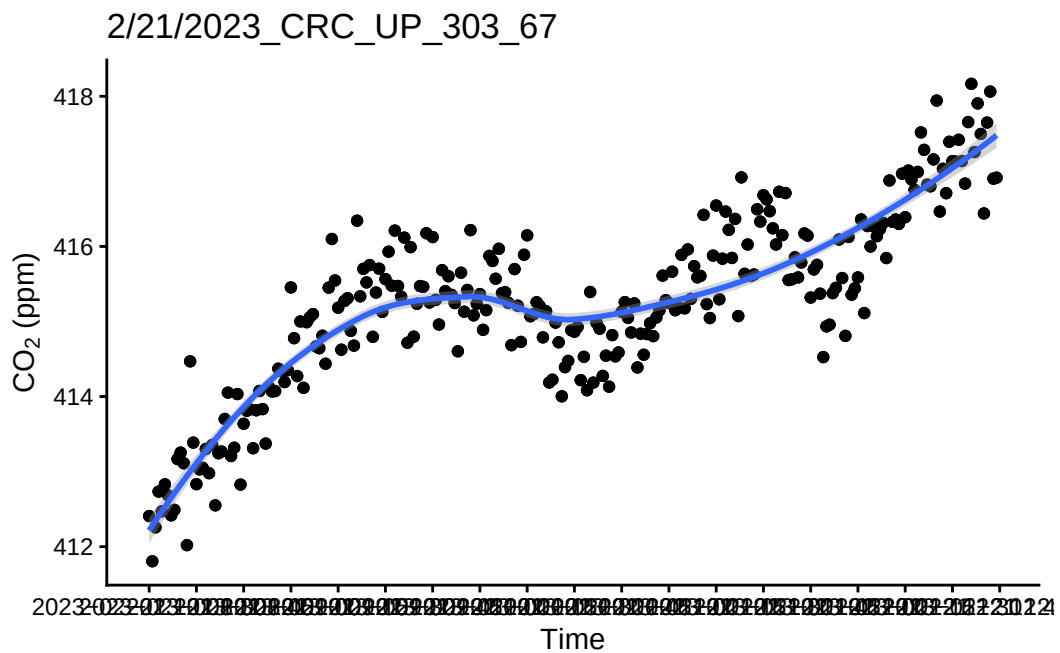
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



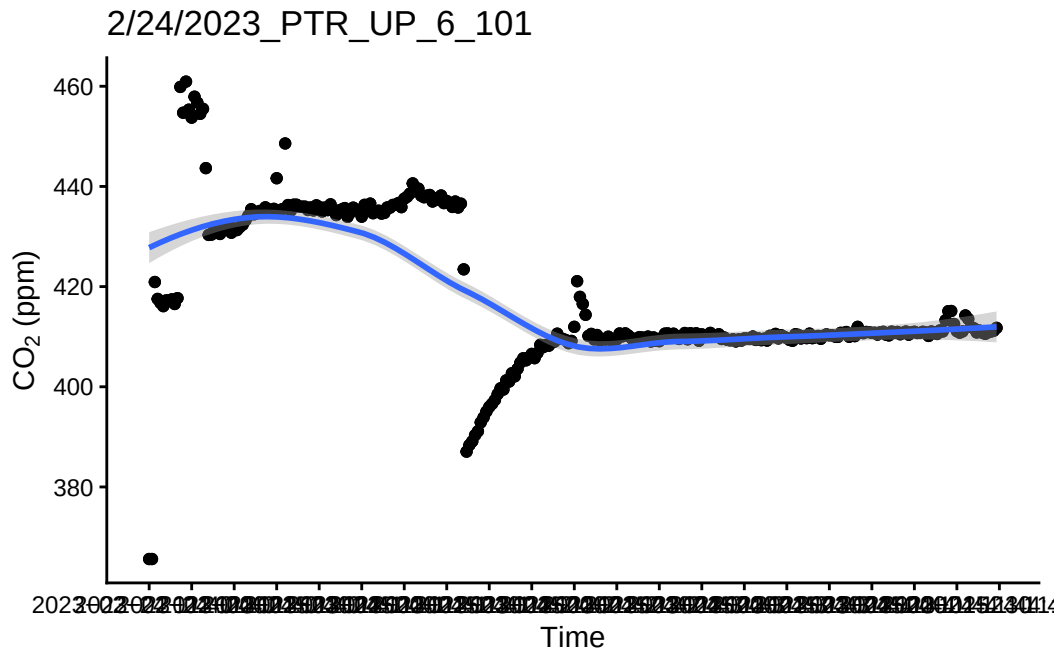
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



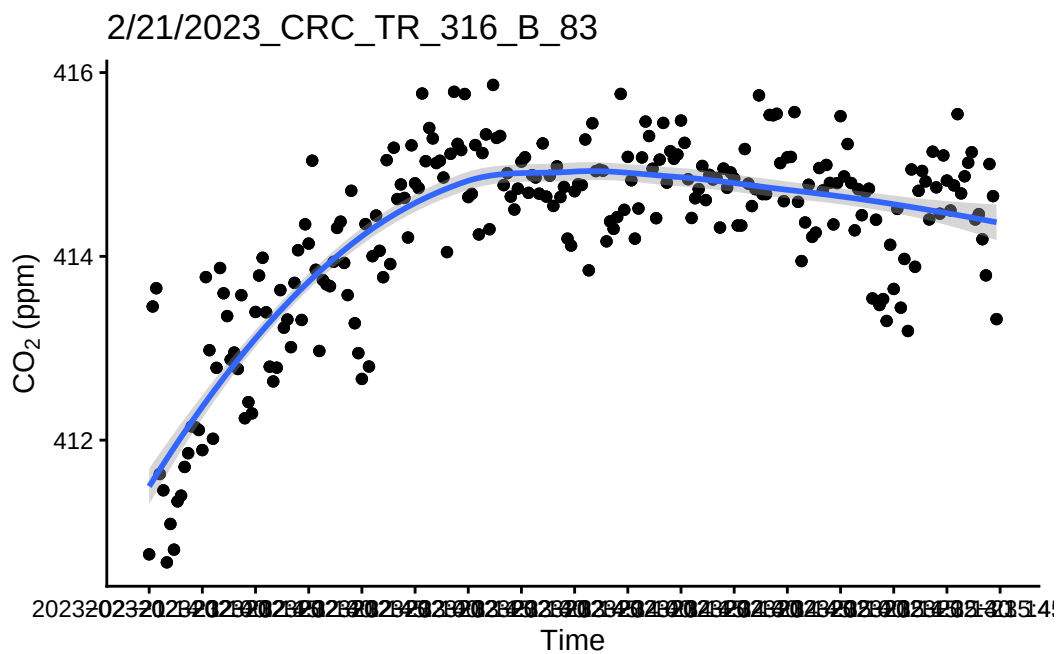
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



```
#####
```

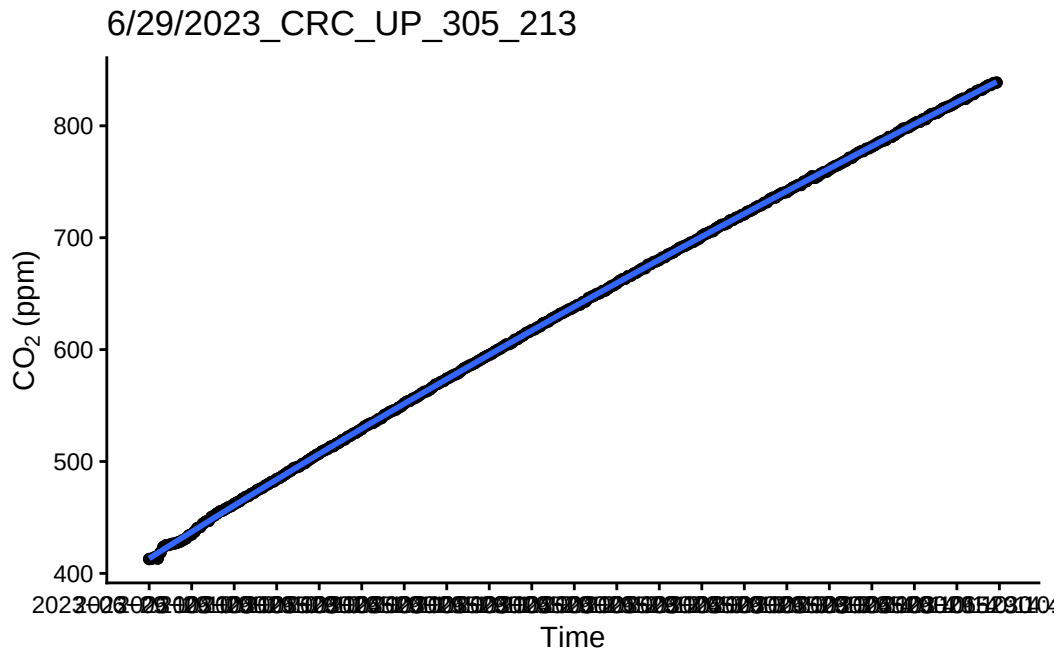
14 Look at the really high or really low ones to see if there is a timing issue or something

```
### pull out fluxes in the 5% and 95% tails to look at really high and really low
tail_plots <- unique(tails$Plot)

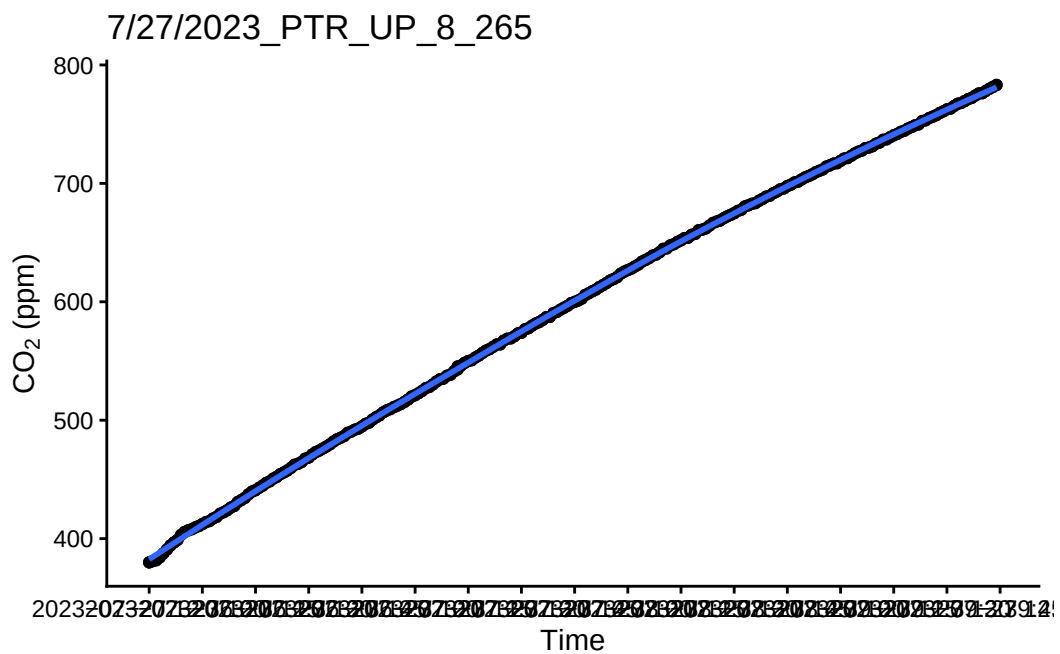
# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
tail_sample <- sample(tail_plots, 2, replace=FALSE)

#Loop to run five random plots subset above
for(P in tail_sample){
  r1 <- subset(alldat_matched , Plot == P & !is.na(metadata_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CO2)) + geom_point() + theme_classic() +
    scale_x_datetime(breaks = "15 sec") +
    ylab(expression(paste( CO [2], " (ppm)"))) +
    xlab("Time") + labs(title = P) +
    geom_smooth()
  print(p1)
}
```

`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

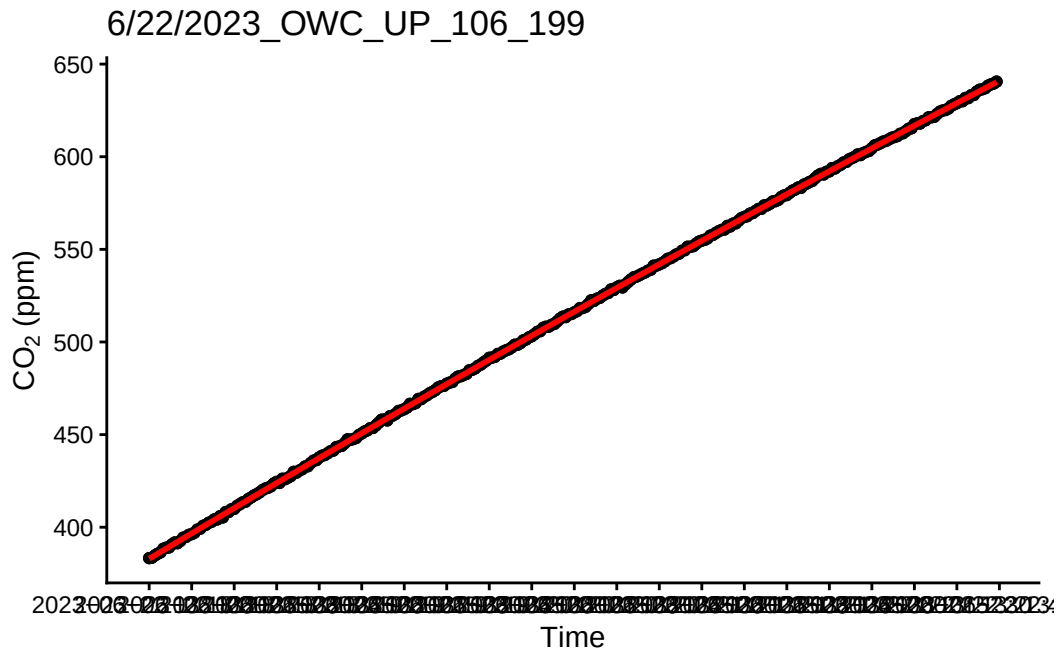


```
#would be nice to put on the plot the flux estimate and if it is a high or low or something  
#####
```

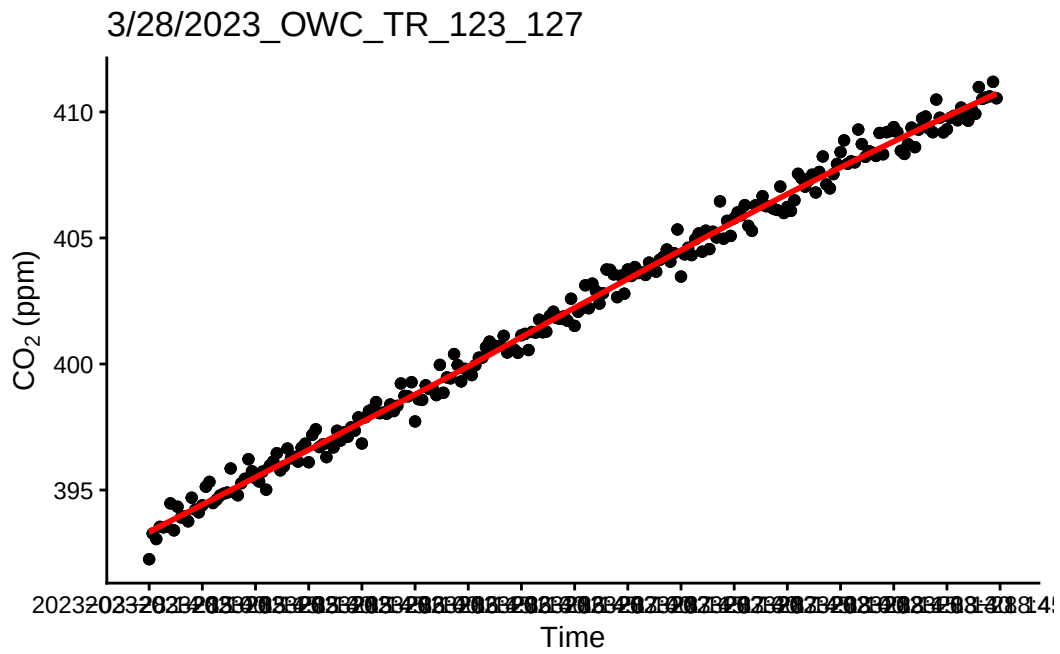
15 Check all fluxes

```
##Put all fluxes together  
fluxes_all <- as.data.frame(subset(fluxes, select = Plot:TIMESTAMP_max))  
  
all_plots <- unique(fluxes_all$Plot)  
  
# take a random sample (FALSE so we don't get the same plot twice)  
# set.seed(...) to guarantee reproducibility  
all_sample <- sample(all_plots, 5, replace=FALSE)  
  
#Loop to run five random plots subset above  
for(P in all_sample){  
  r1 <- subset(alldat_matched, Plot== P & !is.na(metadata_row))  
  p1 <- ggplot(r1, aes(TIMESTAMP, CO2)) + geom_point() + theme_classic() +  
    scale_x_datetime(breaks = "15 sec") +  
    ylab(expression(paste( CO [2], " (ppm)"))) +  
    xlab("Time") + labs(title = P) +  
    geom_smooth(color="red")  
  print(p1)  
}
```

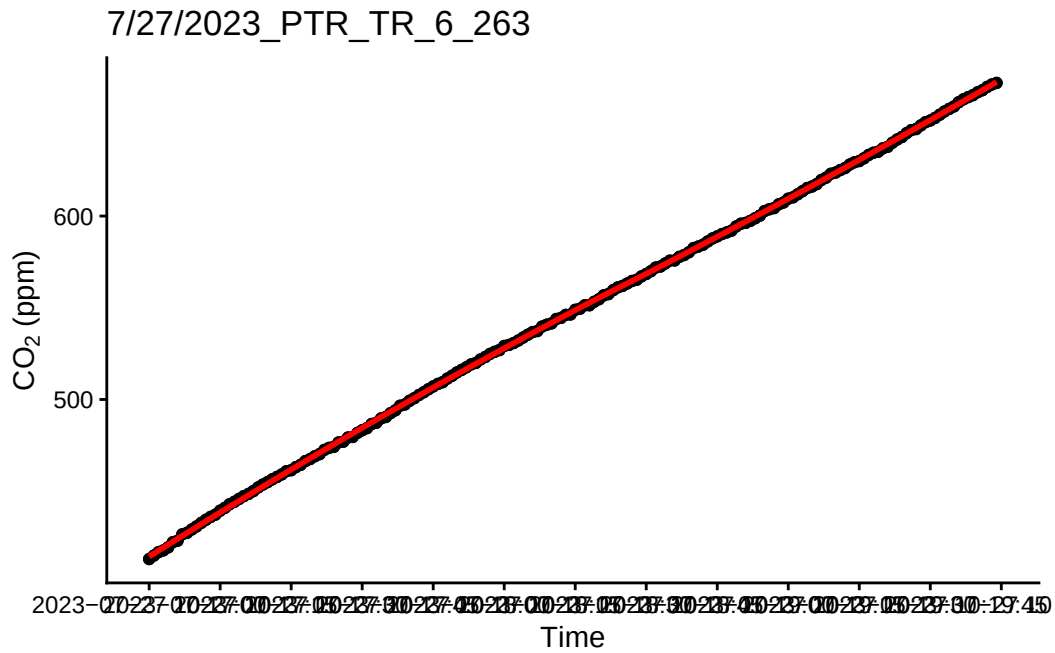
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



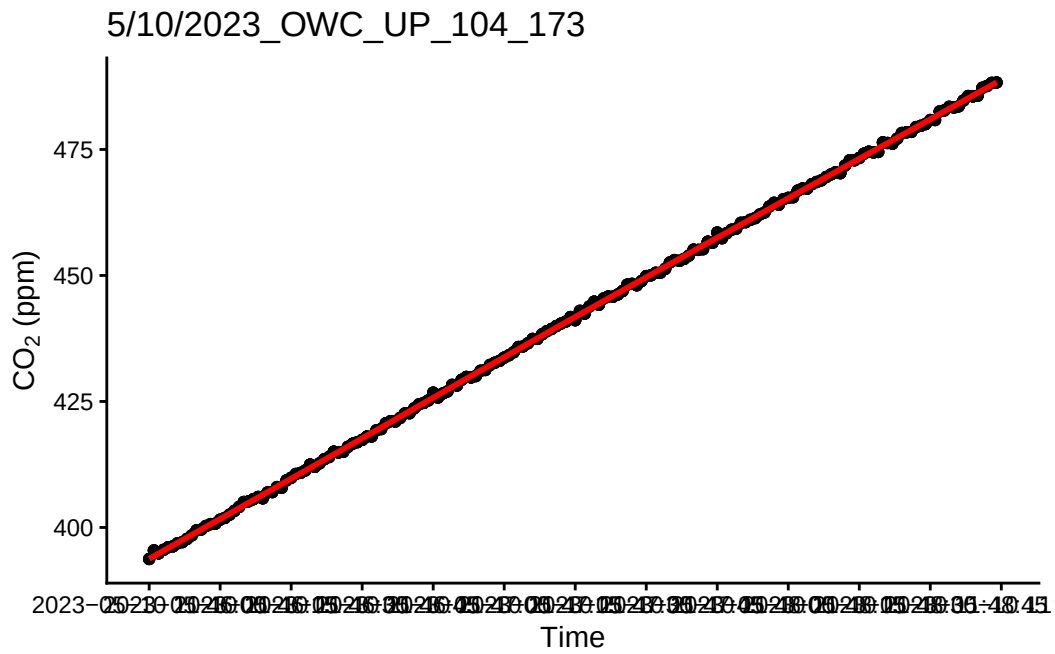
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



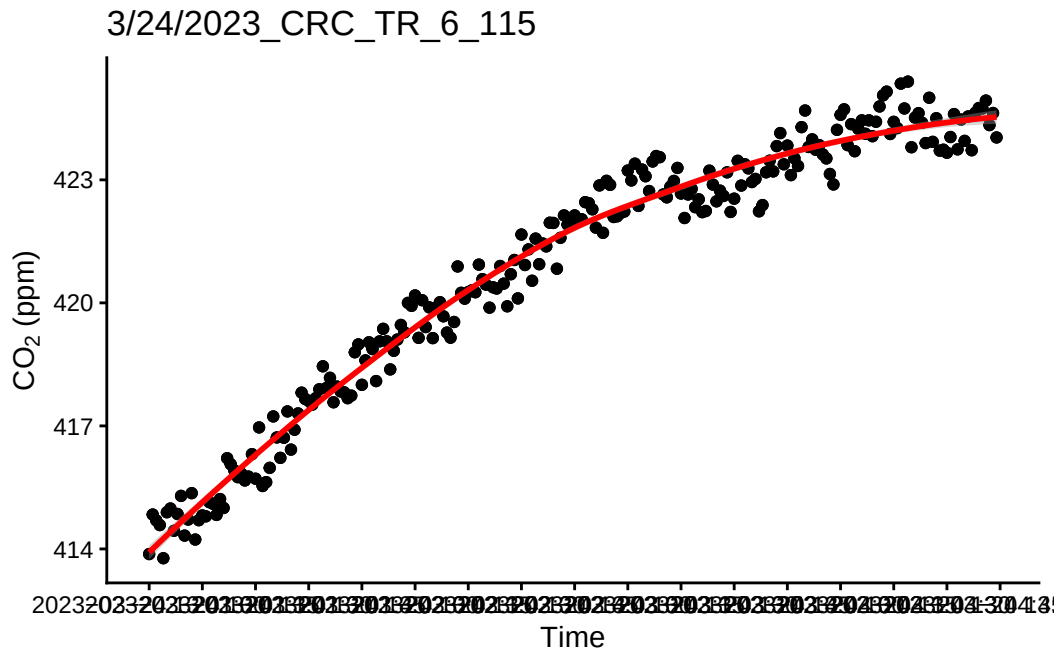
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



```
#####
```

16 Remove any fluxes that do not fit criteria

```
# Create a new dataframe by removing rows that were present in fluxes_weird (11 obs)
#allflux_filtered <- allflux[!(allflux$Plot %in% fluxes_weird$Plo), ] # not all of them are l

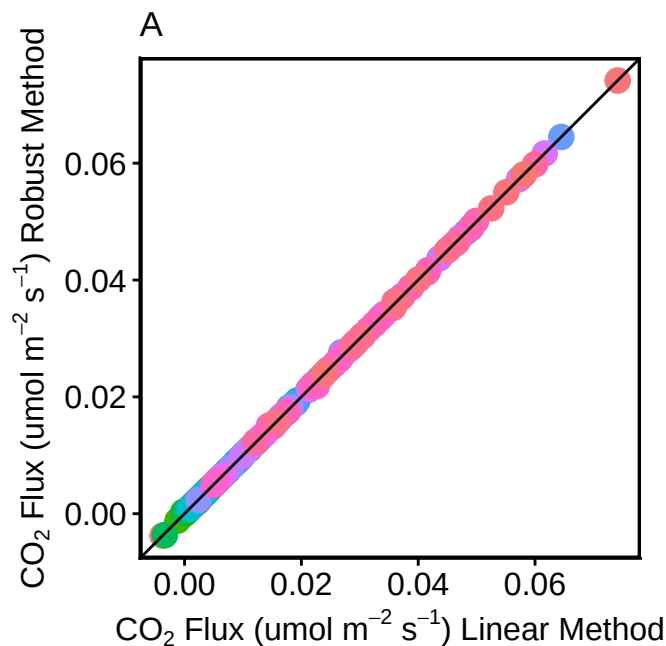
#Any others that are not in the weird category that need to be removed:
#also removing ones that are crazy off but were not in the fluxes_weird dataframe

allflux_filtered <- fluxes_all #>%
#   filter(!Plot %in% c(""))
#
#
# # View the result
# head(allflux_filtered)
```

17 Do some visualization of QAQC plots to assess fluxes

```
p1 <- ggplot(fluxes, aes(lin_flux.estimate, rob_flux.estimate, color = Plot)) +
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  labs(title = "A") +
  ylab(expression(paste( CO [2], " Flux (umol m-2* " s-1*) Robust Method"))) +
  xlab(expression(paste( CO [2], " Flux (umol m-2* " s-1*) Linear Method"))) +
  guides(color = guide_legend(title = "Plot")) +
  theme(legend.position="none") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),
        panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
        aspect.ratio = 1)
```

p1



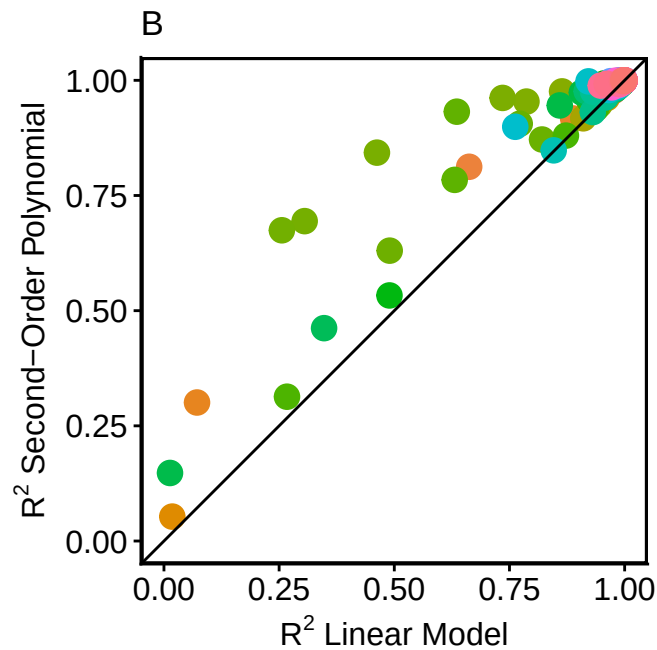
```
p2 <- ggplot(fluxes, aes(lin_r.squared, poly_r.squared, color = Plot)) +
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  xlim(0,1) + ylim(0,1) +
  labs(title = "B") +
```

```

ylab(expression(paste("R"2* " Second-Order Polynomial")))) +
xlab(expression(paste("R"2* " Linear Model")))) +
guides(color = guide_legend(title = "Plot_ID")) +
theme(legend.position="none") +
theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
      axis.title.y = element_text(size=12), legend.text=element_text(size=12),
      panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
      aspect.ratio = 1)

```

p2



```

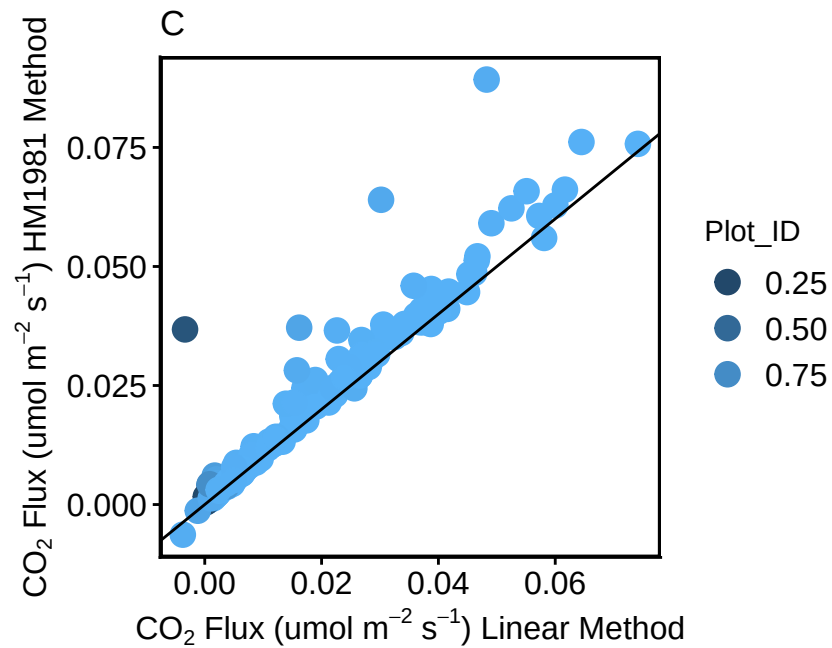
p3 <- ggplot(fluxes, aes(lin_flux.estimate, HM81_flux.estimate, color = lin_r.squared )) +
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  #xlim(0,50) + #ylim(0,50) +
  labs(title = "C") +
  ylab(expression(paste( CO [2], " Flux (umol m"-2* " s"-1*) HM1981 Method")))) +
  xlab(expression(paste( CO [2], " Flux (umol m"-2* " s"-1*) Linear Method")))) +
  guides(color = guide_legend(title = "Plot_ID")) +
  theme(legend.position="right") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),
        panel.border = element_rect(colour = "black", fill=NA, linewidth =1),

```

```
aspect.ratio = 1)
```

p3

Warning: Removed 75 rows containing missing values or values outside the scale range (``geom_point()``).



```
#ggarrange(p1, p2, p3, ncol=3, nrow=1, common.legend = TRUE, legend="none")
```

18 Overall checks fo r2

```
n_total <- nrow(fluxes_all)

r2_summary <- fluxes_all %>%
  mutate(
    lin_bad   = lin_r.squared < 0.9 | is.na(lin_r.squared),
    hm81_bad  = HM81_r.squared < 0.9 | is.na(HM81_r.squared),
    poly_bad  = poly_r.squared < 0.9 | is.na(poly_r.squared),
    fail_all_models = lin_bad & hm81_bad & poly_bad,
```

```

    pass_at_least_one = !(fail_all_models),
    neg_CO2 = lin_flux.estimate < 0 | is.na(lin_flux.estimate)
  ) %>%
  summarise(
    total_obs = n(),
    n_lin_r2_below_0.9 = sum(lin_bad, na.rm = TRUE),
    pct_lin_r2_below_0.9 = 100 * n_lin_r2_below_0.9 / total_obs,
    n_fail_all_models = sum(fail_all_models, na.rm = TRUE),
    pct_fail_all_models = 100 * n_fail_all_models / total_obs,
    n_pass_at_least_one = sum(pass_at_least_one, na.rm = TRUE),
    pct_pass_at_least_one = 100 * n_pass_at_least_one / total_obs,
    n_neg_CO2 = sum(neg_CO2, na.rm = TRUE),
    pct_neg_CO2 = 100 * n_neg_CO2 / total_obs
  )

r2_summary

```

```

total_obs n_lin_r2_below_0.9 pct_lin_r2_below_0.9 n_fail_all_models
1      273              23          8.424908              16
pct_fail_all_models n_pass_at_least_one pct_pass_at_least_one n_neg_CO2
1           5.860806              257          94.13919              6
pct_neg_CO2
1      2.197802

```

19 Read out final dataframe and export for use elsewhere

```

allflux_filtered_renamed <- fluxes %>%
  mutate(
    Date = mdy(str_extract(Plot, "[0-9]+/[0-9]+/[0-9]{4}")),
    Month = month.abb[month(Date)],
    Site = str_extract(Plot, "(PTR|OWC|CRC)"),
    Zone = str_extract(Plot, "_TR_|_UP_") %>%
      str_remove_all("_"),
    Replicate = as.integer(
      str_extract(Plot, "(?<=_ (TR|UP)_)[0-9]+")
    ),
    Plot_ID = as.integer(
      str_extract(Plot, "[0-9]+$") ) %>%

```

```

# Optional: rebuild a clean Plot label
mutate(
  Plot_clean = paste(
    format(Date, "%Y-%m-%d"),
    Site,
    Zone,
    Replicate,
    Plot_ID,
    sep = "_"
  )
) %>%

select(
  Plot_clean,
  Date,
  Month,
  Site,
  Zone,
  Replicate,
  Plot_ID,
  everything(),
  -Plot
)

#export as a csv to the folder that you want
write.csv(allflux_filtered_renamed, file="Processed Data/COMPASS_Synoptic_LE_TreeStem_Fluxes.csv", as.is=T)
# #Change file name

```

20